

Smart Contract Engineer, Practical Exercise

Protocol engineering role · Take-home practical

Time	One hour. Treat it as a hard budget, not a target to beat. How you spend it is part of what we are assessing.
Deadline	Within 48 hours of receipt. Schedule the hour whenever suits you.
Submit	A GitHub repo link, in a single reply. Nothing else.
AI tools	Encouraged. This role expects you to build with AI in the loop, so use the tooling you would use on the job. We only require that you can explain and defend every decision you ship.

The context

You have joined the team that maintains a single-collateral money market. Suppliers deposit a base asset and earn interest, borrowers post collateral and draw against it, and positions that fall below their borrowing power can be liquidated.

The market is live. It is deployed behind a transparent upgradeable proxy and it holds real user balances. The existing test suite is green. An audit has been scheduled, and your job is to get there first.

The contract has defects. Finding them is the exercise.

What you are given

- A self-contained Foundry project. `forge build` and `forge test` work out of the box, no network access needed, `forge-std` is vendored in `lib/`.
- `src/LendingMarket.sol`, the implementation and the only file you need to change. Roughly 250 lines.
- `src/TransparentUpgradeableProxy.sol`, the proxy the market is deployed behind.
- `script/Deploy.s.sol`, the staging deployment, which mirrors the production topology.
- `test/LendingMarket.t.sol`, the suite the market ships with today. It covers the happy paths and passes. Treat it as a starting point, not a specification.
- The repo README explains the accounting model, the scaling convention and the role split. Read it first. It is there so you spend your hour on judgment rather than on archaeology.

What to do

1. **Find the defects.** There are at least three serious ones: anything that could cost users or the protocol money, or put the market into a state it cannot recover from. Finding two and explaining them precisely beats a list of five you cannot defend.
2. **Fix them.** Your fix has to be shippable to the live proxy, which already holds user balances.

3. **Prove each fix.** Add a test that fails against the original code and passes against yours. Plain unit tests are fine.
4. **Write it up.** A short findings section in the README: what you found, how severe, how it would be exploited, and what you changed. One or two sentences each is plenty.

Two constraints, so our verification harness can run against your work:

- Keep the contract at `src/LendingMarket.sol` and keep its name.
- Keep the existing external function signatures as they are. You are free to add new ones.

If you run out of time

You probably will, and that is fine. An hour is not enough to do everything you would want to do before an audit, which is deliberate.

A partial submission with clear reasoning beats a rushed complete one. If you found something and did not get to fixing it, say so and describe the fix you would write. If you were unsure whether something was a defect, say that too, and say what you would check next.

A short scope note

Three or four lines in your README, no more:

- What you prioritized, and what you deliberately left out.
- Where you were unsure, and what you would verify with more time.
- Whether you used AI tooling, and one suggestion you validated, modified or rejected.

What we care about

We are looking at your judgment as much as your output: what you choose to look at first, how you reason about severity, whether your fix is something that could actually be shipped to a live market, and how clearly you explain your decisions.

To submit: reply with your GitHub repo link. Public, or invite us. Your normal working commit history is helpful review context, but we care more about whether you understand and can defend the submitted result.